



958225 Digital Game Production

แบบฝึกปฏิบัติการ 8

(LAB 8)

หัวข้อ: Project Planning

ชื่อ.....

รหัส.....

Section (LEC)

สาขาวิชาดิจิทัลเกม

วิทยาลัยศิลปะ สื่อ และเทคโนโลยี

มหาวิทยาลัยเชียงใหม่

วิชา: DG225 Digital Game Production | สัปดาห์ที่: 8 | เวลา: 2 ชั่วโมง 10 นาที

ใน Lab 07 ทีมสร้าง sprint-01.md ไว้เป็นโครงเปล่าพร้อม mock backlog 2 แถว ในแลปนี้ทีมจะทำ **Sprint Planning จริง** ผ่านเอกสาร 3 ไฟล์ตามลำดับ: `01-product-backlog` (รวม User Story ทั้งหมด) → `02-sprint-backlog` (จัดสรร Story ลง 4 Sprint ตลอดเทอม) → `sprint-plan-[NN]` (แผนละเอียด + แยก Task ของ Sprint ที่กำลังทำ)

📁 โครงสร้าง Repository ของ Lab นี้

เพิ่ม/แก้ไขไฟล์เหล่านี้ใน group repo:

```
[project-name]-game/
├── docs/
│   └── agile/
│       ├── 01-product-backlog.md    ← สร้างในหัวข้อที่ 1
│       ├── 02-sprint-backlog.md     ← สร้างในหัวข้อที่ 2
│       └── sprint-plan-01.md        ← เปลี่ยนชื่อจาก sprint-01.md (Lab 07) + อัปเดตในหัวข้อที่ 3-4
```

📄 ไฟล์ประกอบ

ดาวน์โหลด template ทั้ง 3 ไฟล์นี้จาก **ไฟล์แนบใน LMS** คู่กับใบงาน Lab นี้ แล้วคัดลอกไปใช้ตามหัวข้อที่ระบุ — แต่ละ template มีตัวอย่างที่กรอกไว้แล้วในไฟล์เลย ไม่ต้องเปิดดูตัวอย่างซ้ำในใบงานนี้อีก:

- product-backlog-template.md — ใช้ในหัวข้อที่ 1
- sprint-backlog-template.md — ใช้ในหัวข้อที่ 2
- sprint-plan-template.md — ใช้ในหัวข้อที่ 3 (และ Sprint 2-4 ในแลปถัดไป)

🚩 **โปรเจกต์นี้วางแผนแบ่งงานตลอดเทอมเป็น 4 Sprint — 02-sprint-backlog.md** เก็บภาพรวมทั้ง 4 Sprint ไว้ทีเดียว ส่วน sprint-plan-[NN].md มีแยกทีละ Sprint (sprint-plan-01.md – sprint-plan-04.md) Lab นี้ทำเฉพาะ **Sprint 1** เท่านั้น ส่วน Sprint 2-4 จะทำในแลปถัดไปด้วยขั้นตอนเดียวกัน โดยคัดลอกจาก template sprint-plan-template.md อีกครั้ง

📌 เตรียมตัวก่อนเข้า Lab

ตรวจสอบว่าพร้อมก่อนเริ่ม:

- ☐ docs/agile/sprint-01.md จาก Lab 07 เสร็จและ push แล้ว (มี Sprint Goal เบื้องต้น)
- ☐ docs/gdd/ ครบและเป็นฉบับล่าสุด — จะใช้เป็นแหล่งอ้างอิงฟีเจอร์ทั้งหมด
- ☐ ทุกคน git pull ล่าสุดจาก group repo แล้ว
- ☐ แต่ละคนรู้ว่าตัวเองมีเวลาทำงานเกมที่ชั่วโมงต่อสัปดาห์ (ใช้คำนวณ capacity ในหัวข้อ 2)

หัวข้อการฝึกปฏิบัติที่ 1 — 01-product-backlog: สร้าง Product Backlog จาก GDD

วัตถุประสงค์: แปลงฟีเจอร์ทั้งหมดใน GDD ให้เป็น User Story ที่เขียนในรูปแบบมาตรฐาน จัดกลุ่มด้วย MoSCoW พร้อม Acceptance Criteria เพื่อให้มี backlog กลางที่ทุก Sprint ในอนาคตหยิบงานไปทำได้ ใช้เวลาปฏิบัติประมาณ 35 นาที

1.1 คัดลอก Template และทำความเข้าใจรูปแบบ User Story

คัดลอกจาก template product-backlog-template.md (ไฟล์แนบใน LMS) มาสร้างไฟล์ docs/agile/01-product-backlog.md โดยตรง (ไม่ต้องร่างในไฟล์แยก) — template มีตัวอย่างที่กรอกแล้วในแต่ละกลุ่ม MoSCoW ให้ดูเป็นแนวทาง

ทุก Story เขียนในรูปแบบเดียวกันเสมอ: As a [บทบาท], I want [สิ่งที่ต้องการ], so that [ประโยชน์ที่ได้] — Story บอกแค่ **ใครได้ประโยชน์อะไร** ยังไม่ต้องบอกวิธีทำ (implementation) จะไปกำหนดในขั้นตอน Task Breakdown ที่หัวข้อ 3)

1.2 ระดมฟีเจอร์จาก GDD เขียนลง Product Backlog โดยตรง

เปิด docs/gdd/ ทั้งหมด แล้วช่วยกันไล่ทีละหน้า แปลงทุกฟีเจอร์ที่เจอให้เป็น User Story แล้วเพิ่มเป็นแถวใหม่ในกลุ่ม MoSCoW ที่เหมาะสม (Must Have / Should Have / Nice to Have) พร้อมใส่ Acceptance Criteria และ Estimate (SP) คร่าวๆ ไปพร้อมกัน — ไม่ต้องสมบูรณ์แบบตั้งแต่รอบแรก ปรับกลุ่มหรือตัวเลขที่หลังได้เสมอ

Acceptance Criteria คือเงื่อนไขสั้นๆ ที่ตรวจสอบได้จริง (testable) ว่า Story นี้ "เสร็จ" เมื่อไหร่ — เขียน 1 ประโยคพอ ถ้าตรวจไม่ได้ว่าเสร็จหรือยัง (เช่น "เล่นสนุก") แปลว่ายังเขียนไม่ดีพอ

>

Estimate (SP) คือ Story Point — ตัวเลขเปรียบเทียบ "ความใหญ่/ความซับซ้อน" ของ Story หนึ่งเทียบกับอีก Story หนึ่ง (ไม่ใช่ชั่วโมงทำงานตรงๆ) ใช้เลขง่ายๆ เช่น 1, 2, 3, 5, 8 — ยิ่งเลขสูงยิ่งใหญ่/ซับซ้อนกว่า ที่ประเมินโดยเทียบ Story ที่เคยรู้ขนาดกันเองในกลุ่ม ไม่ต้องแม่นยำ

>

คอลัมน์ Sprint ยังไม่ต้องกรอกตอนนี้ — จะย้อนกลับมาใส่หลังทำหัวข้อ 2 เสร็จ

หัวข้อการฝึกปฏิบัติที่ 2 — 02-sprint-backlog: วางแผนการจัดสรร Story ทั้ง 4 Sprint

วัตถุประสงค์: ประเมิน capacity ของทีมแล้วจัดสรร User Story จาก Product Backlog ลง Timeline ทั้ง 4 Sprint ของโปรเจกต์ เพื่อเน้นฝึกทักษะวางแผนและ estimate มากกว่าการสร้างโครงสร้างเอกสารซ้ำทุกสัปดาห์
ใช้เวลาปฏิบัติประมาณ 30 นาที

2.1 ประเมิน Team Capacity ต่อ Sprint

รวมชั่วโมงทำงานที่แต่ละคนมีจริงต่อ 2 สัปดาห์ (นอกเวลาเรียน):

$\text{Capacity} = \text{จำนวนคน} \times \text{ชั่วโมงเฉลี่ยต่อคนต่อ Sprint (2 สัปดาห์)}$

ตัวอย่าง: ทีม 4 คน แต่ละคนมีเวลาเฉลี่ย 12 ชม./2 สัปดาห์ → Capacity = 48 ชม./Sprint — ใช้ตัวเลขนี้กะปริมาณ Story ที่หยิบเข้าแต่ละ Sprint อย่าหยิบเกินจนรู้อยู่แล้วว่าทำไม่ทัน

2.2 สร้าง Sprint Backlog และจัดสรร Story ลง 4 Sprint

คัดลอกจาก template sprint-backlog-template.md (ไฟล์แนบใน LMS) มาสร้างไฟล์ docs/agile/02-sprint-backlog.md — ไฟล์นี้มี Timeline (ตาราง + Mermaid Gantt) และตารางจัดสรร Story ของ Sprint 1-4 พร้อมตัวอย่างที่กรอกแล้วให้ดูเป็นแนวทาง

ไล่จาก 01-product-backlog.md เอา Story ใน **Must Have (MVP) ก่อนเสมอ** ใส่ Sprint 1 จนกว่า Story Point รวมจะใกล้เต็ม capacity ที่ประเมินไว้ แล้วไล่ Story ที่เหลือ (Must Have ที่ยังไม่ได้ทำ → Should Have → Nice to Have) กระจายลง Sprint 2-4 แบบคร่าวๆ

Sprint 1 คือของจริง ใช้ทำงานทันที — Sprint 2-4 เป็นแค่ draft ระดับ release plan ปรับได้เสมอเมื่อเข้าใจงานมากขึ้น เป้าหมายของหัวข้อนี้คือฝึกกะปริมาณ SP ต่อ Sprint ให้ใกล้เคียง capacity ไม่ใช่ล็อก scope ตายตัว

2.3 เชื่อมข้อมูลกลับไป Product Backlog

กลับไป 01-product-backlog.md กรอกคอลัมน์ Sprint ของทุก Story ให้ตรงกับ Sprint ที่จัดสรรไว้ใน 02-sprint-backlog.md (ไล่เลข 1-4)

ถ้าเลข Sprint ในทั้งสองไฟล์ตรงกันทุก Story แสดงว่าเชื่อมข้อมูลถูกต้องแล้ว

หัวข้อการฝึกปฏิบัติที่ 3 — sprint-plan-01: สร้าง Sprint Plan และแตก Task

วัตถุประสงค์: สร้างแผนละเอียดของ Sprint 1 พร้อมแตกแต่ละ User Story ให้เป็น Task ย่อยที่ทำเสร็จได้ภายใน 1 วันหรือน้อยกว่า เพื่อให้ลงมือทำได้ทันทีโดยไม่ต้องตีความเพิ่ม ใช้เวลาปฏิบัติประมาณ 45 นาที

3.1 เปลี่ยนชื่อไฟล์และสร้าง Sprint Plan

คัดลอกโครงจาก template `sprint-plan-template.md` (ไฟล์แนบใน LMS) มาแทนที่เนื้อหาทั้งหมดในไฟล์ กรอก **Sprint Goal** ให้เจาะจง (ไม่ใช่ placeholder จาก Lab 07 แล้ว) แล้วนำ Story ของ Sprint 1 จาก `02-sprint-backlog.md` มาใส่ในตาราง Sprint Backlog พร้อมระบุคนรับผิดชอบและ Status

ตรวจว่า Story ในตาราง Sprint Backlog ของ `sprint-plan-01.md` ตรงกับแถว Sprint 1 ใน `02-sprint-backlog.md` และตรงกับ Story ที่ mark `Sprint: 1` ใน `01-product-backlog.md` — ทั้ง 3 ไฟล์ต้องสอดคล้องกัน

3.2 ทำไมต้องแตก Story เป็น Task

User Story บอกแค่ "ผลลัพธ์ที่ผู้เล่นได้" แต่ยังใหญ่เกินกว่าจะลงมือทำได้ในคราวเดียว — ต้องแตกเป็น Task ที่บอก "ขั้นตอนที่คนต้องทำจริง"

ตัวอย่าง — Story "As a player, I want to jump" แตกเป็น:

1. ออกแบบ input mapping สำหรับปุ่มกระโดด
2. เขียน physics/velocity สำหรับการกระโดด
3. สร้าง jump animation (หรือ placeholder sprite)
4. ทดสอบกระโดดชนขอบแพลตฟอร์มแล้ว behavior ถูกต้อง

เช็คว่า Task เล็กพอหรือยัง: ถ้า Task เดียวใช้เวลาทำเกิน 1 วัน หรืออธิบายเป็นประโยคเดียวไม่ได้ว่า "ทำอะไรเสร็จแบบไหน" — แยกต่อให้เล็กลงอีก

3.3 กรอก Task List ใน Sprint Plan

เติมรายละเอียดใน section ## Tasks ที่มีอยู่แล้วในไฟล์ (คัดลอกมาจาก template ในข้อ 3.1) ให้ครบทุก Story ในตาราง Sprint Backlog — ใช้ **Dataview inline field** เพื่อ track owner/estimate/status ของแต่ละ Task ได้ในอนาคต ทำซ้ำ 1 block ต่อ 1 Story

เช็คว่า: ผลรวม estimate ของ Task ทั้งหมดต่อ Story ควรใกล้เคียงกับ Story Point ที่ประเมินไว้ในหัวข้อ 1 — ถ้าห่างกันมาก (เช่น Story ประเมินไว้ 3 SP แต่ Task รวมกันปาไป 15 ชม.) แสดงว่าประเมิน Story Point ตอนแรกคลาดเคลื่อน กลับไปปรับใน `01-product-backlog.md` และ `02-sprint-backlog.md` ด้วย

หัวข้อการฝึกปฏิบัติที่ 4 — มอบหมาย Task แรกและเริ่มลงมือทำ

วัตถุประสงค์: แต่ละคนเลือก Task แรกที่จะเริ่มทำทันทีหลังแลป แล้ว commit เอกสารที่อัปเดตแล้วเข้า repo เพื่อให้ทีมเริ่ม Sprint ได้จริงตั้งแต่วันนี้ ใช้เวลาปฏิบัติประมาณ 15 นาที

4.1 เลือก Task แรกของแต่ละคน

แต่ละคนเลือก 1 Task จาก ## Tasks ใน sprint-plan-01.md

ที่ตัวเองรับผิดชอบและจะเริ่มทำทันทีหลังแลป เปลี่ยน [status:: todo] เป็น [status:: doing] เฉพาะ Task นั้น

4.2 ยืนยันเอกสารครบตามที่ตกลง

ก่อน commit ตรวจสอบว่า:

- 01-product-backlog.md ทุก Story มีคอลัมน์ Sprint ครบ
- 02-sprint-backlog.md มี Timeline + การจัดสรร Story ครบทั้ง 4 Sprint
- sprint-plan-01.md มี Sprint Goal ที่เจาะจง, Sprint Backlog ตรงกับ Sprint 1 ใน 02-sprint-backlog.md, ทุก Story มี Task ย่อยพร้อม owner และ estimate
- มีอย่างน้อย 1 Task ต่อคนที่สถานะเป็น doing

ถ้าครบทุกข้อ แสดงว่า Sprint นี้พร้อมเริ่มทำงานจริง — ไม่ต้องกลับมาประชุมตีความ scope อีกระหว่างสัปดาห์

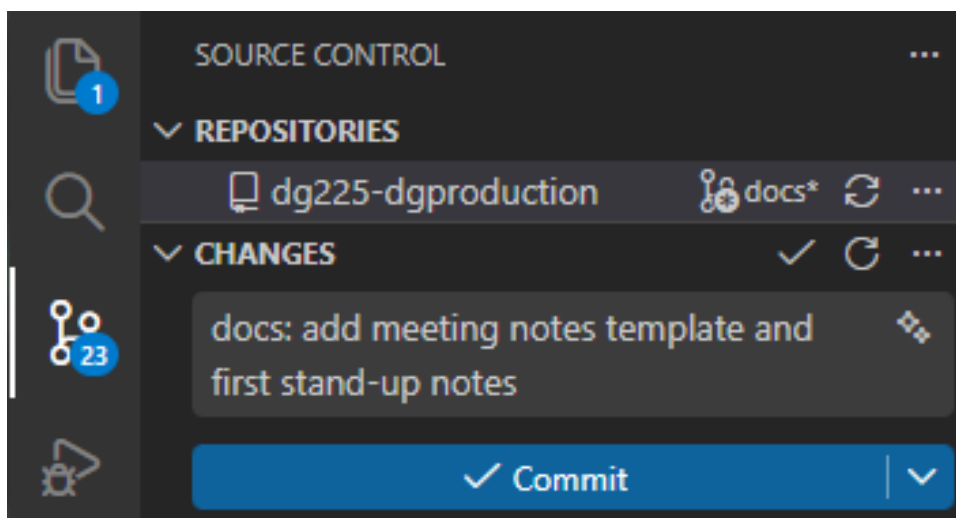
✓ Checklist ก่อนออกจาก Lab

- ☐ docs/agile/01-product-backlog.md มี User Story ครบจากพีเจอาร์ใน GDD จัดกลุ่มแบบ MoSCoW พร้อม Acceptance Criteria, Estimate (SP) และคอลัมน์ Sprint
- ☐ docs/agile/02-sprint-backlog.md มี Timeline ทั้ง 4 Sprint (ตาราง + Mermaid Gantt) และการจัดสรร Story ครบทั้ง 4 Sprint
- ☐ docs/agile/sprint-plan-01.md (เปลี่ยนชื่อจาก sprint-01.md) มี Sprint Goal ที่เจาะจง และ Sprint Backlog ตรงกับ Sprint 1 ใน 02-sprint-backlog.md
- ☐ ทุก User Story ใน Sprint 1 ถูกแตกเป็น Task ย่อยพร้อม owner/estimate/status
- ☐ แต่ละคนมี Task อย่างน้อย 1 รายการที่สถานะเป็น doing และรู้ว่าตัวเองต้องเริ่มทำอะไรทันทีหลังแลป

Git Commit

วัตถุประสงค์: บันทึกงานของ Lab นี้เข้า Git ด้วย commit message ที่สื่อความหมาย แล้ว push ขึ้น GitHub เพื่อเตรียมผลงาน ตัวอย่างดังนี้:

```
"docs: add product backlog, sprint backlog, and sprint 1 plan with task breakdown"
```



วิธีการ commit การเปลี่ยนแปลงด้วย command line (Optional)

หมายเหตุ: commit เฉพาะไฟล์เอกสารของ Lab นี้ และตรวจว่า .gitignore กันไม่ให้ bin/, obj/ หลุดขึ้น remote repo

```
git add docs/agile/01-product-backlog.md docs/agile/02-sprint-backlog.md docs/agile/sprint-plan-01.md
git commit -m "docs: add product backlog, sprint backlog, and sprint 1 plan with task breakdown"
git log --oneline
```

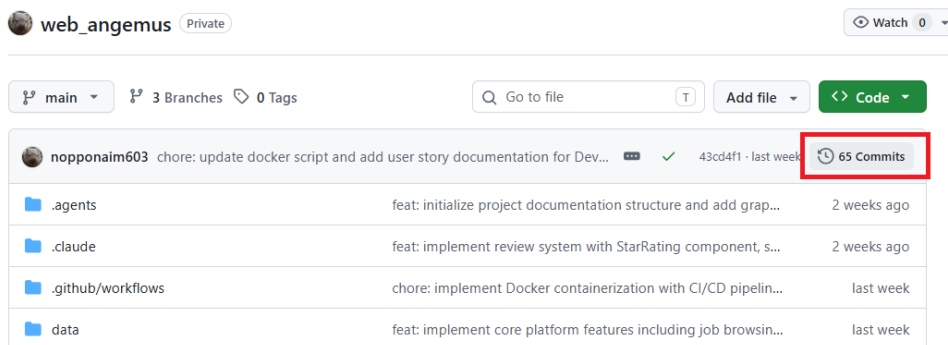


การส่งงาน

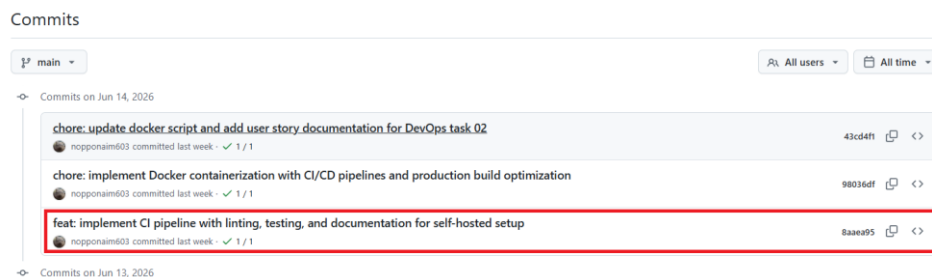
ส่งงาน Lab นี้โดย คัดลอกลิงก์ commit บน GitHub มาวางใน LMS (Mango Canvas)

วิธีหาลิงก์ commit:

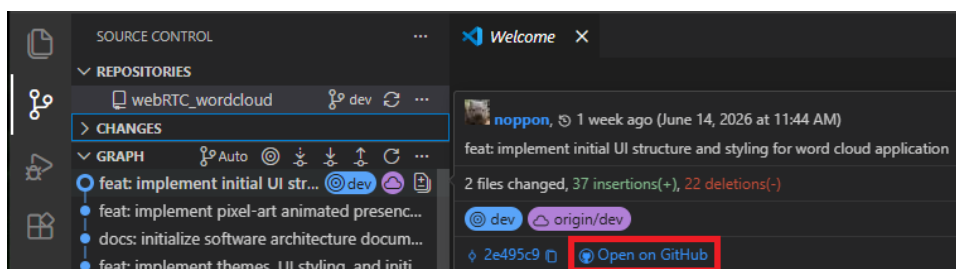
1. เปิด repository ของตัวเองบน GitHub ในเบราว์เซอร์
2. คลิก "Commits" (ไอคอนนาฬิกา 🕒 พร้อมจำนวน commit) ด้านบนของรายการไฟล์



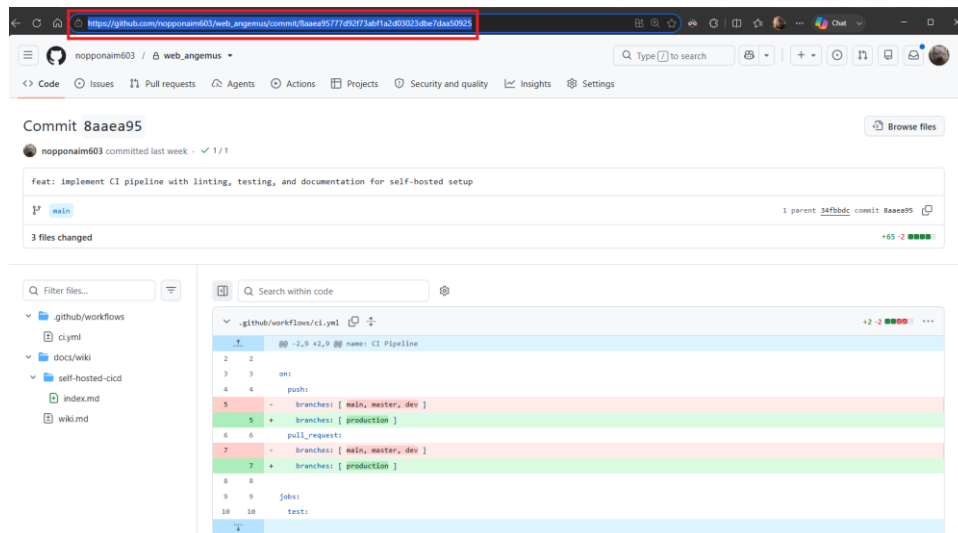
3. คลิกเลือก commit ของ Lab นี้ที่ต้องการส่ง



หรือเปิดจาก VS Code ผ่านหน้าต่าง "SOURCE CONTROL" → ลากเมาส์ไปหยุดเหนือ commit ที่ต้องการในหน้าต่าง GRAPH จะแสดง Popup ข้อมูลของ commit นั้น แล้วกดปุ่ม **Open on GitHub** เพื่อเปิดหน้าการเปลี่ยนแปลงครั้งนั้นในเบราว์เซอร์



4. คัดลอก URL จาก address bar แล้วนำไปวางในช่องส่งงานบน LMS



ตัวอย่าง URL:

`https://github.com/username/[project-name]-
game/commit/abc1234...`

✓ ก่อนส่ง: เปิดลิงก์ในโหมด Incognito เพื่อตรวจสอบว่าเห็นหน้า commit ที่แสดงไฟล์และโค้ดได้จริง